

云计算技术

课程设计任务书

Cloud Computing Technologies — Course Project

课程代码	SCAI004712
授课教师	戴朋林、丛荣
适用专业	软件工程 / 计算机科学与技术
授课对象	大三学生（大三下）
学分/学时	2 学分 / 32 学时
云资源	华为云"智能基座"代金券
更新时间	2026 年春季学期

附加题标识：◆

一、概述.....	4
1.1 设计思路与目标.....	4
1.2 难度分级.....	错误!未定义书签。
1.3 华为云服务速查.....	4
1.4 提交与答辩说明.....	4
二、第一部分：云计算平台搭建（50 分）.....	5
2.1 任务目标.....	5
2.2 任务 1：应用容器化.....	5
2.3 任务 2：CCE 集群搭建.....	5
2.4 任务 3：应用部署.....	6
2.5 任务 4：持久化存储.....	6
2.6 任务 5：ConfigMap Volume 挂载.....	7
2.7 任务 6：HPA 弹性伸缩.....	7
三、第二部分：并行编程实战（40 分）.....	8
3.1 方向选择.....	8
3.2 方向 A：Spark 大数据分析.....	8
A-0. 环境部署（基础级，10 分）.....	8
A-1. 数据清洗（基础级，10 分）.....	8
A-2. Spark SQL 统计分析（进阶级，15 分）.....	8
A-3. 性能对比与 Amdahl 分析（挑战级，5 分）.....	9
3.3 方向 B：MPI 并行科学计算.....	9
B-0. 环境部署（基础级，10 分）.....	9
B-1. 并行算法实现（基础级，10 分）.....	9
B-2. 性能测试与 Amdahl 分析（进阶级，15 分）.....	10
B-3. 非阻塞通信优化（挑战级，5 分）.....	10
四、附加题（最高 +15 分，自愿完成）.....	10
附加题 1：监控系统（+5 分）.....	10
附加题 2：CI/CD 流水线（+5 分）.....	11
附加题 3：前沿专题（+5 分，三选一）.....	11
五、评分标准.....	11
5.1 第一部分评分细则（50 分）.....	11
5.2 第二部分评分细则（40 分）.....	12
5.3 报告质量（10 分）.....	13
5.4 总分汇总.....	13
六、实验报告结构要求.....	13
附录 A：K8s 脚手架模板（第一部分）.....	13
A-1 Dockerfile 模板.....	14
Dockerfile.backend（Python Flask，多阶段构建）.....	14
Dockerfile.frontend（Nginx 静态页）.....	14
docker-compose.yml（本地联调用）.....	14
A-2 K8s 核心资源 YAML 模板.....	15
deployment.yaml（后端 Flask API）.....	15
service.yaml（后端 LoadBalancer + Redis ClusterIP）.....	15
configmap.yaml + secret.yaml.....	16
A-3 PVC 模板（持久化存储）.....	16

A-4 HPA 模板（弹性伸缩）	17
附录 B：并行编程脚手架（第二部分）	17
B-1 方向 A：Spark on K8s 模板	17
sparkapplication.yaml（SparkApplication CR）	17
wordcount.py（PySpark 入门示例，作业提交验证用）	18
B-2 方向 B：MPI on K8s 模板	18
mpijob.yaml（MPIJob CR）	18
pi_mpi.py（蒙特卡洛估算 π ，入门示例，作业提交验证用）	19

一、概述

1.1 设计思路与目标

本课程设计以"理解原理 → 修改模板 → 验证效果"为主线。教师组提供完整脚手架（模板 Dockerfile、K8s YAML、Spark/MPI 作业示例，详见附录），学生在脚手架基础上按要求修改并在华为云平台部署，将课堂概念落地到真实生产环境。

设计分两部分，另设附加题，满分 100 分，附加题最高额外加 15 分：

- 第一部分（50 分）：云计算平台搭建——基于华为云 CCE 搭建 K8s 平台，部署容器化 Web 应用。
- 第二部分（40 分）：并行编程实战——在 K8s 平台上运行 Spark 大数据分析或 MPI 并行计算作业。
- 报告质量（10 分）。附加题（最高+15 分）：监控系统 / CI/CD / 前沿专题。

1.3 华为云服务速查

本课程设计使用以下四项华为云服务，均可通过"智能基座"代金券免费使用，无需自费：

服务名称	全称	作用简介	在本设计中的用途
SWR	软件仓库 SWR (SoftWare Repository)	私有 Docker 镜像仓库，类似于私有 DockerHub，用于存储和管理容器镜像	存放学生构建的后端/前端镜像，K8s 部署时从此处拉取
CCE	云容器引擎 CCE (Cloud Container Engine)	托管的 Kubernetes 服务，华为云帮助管理 Master 节点，学生只需管理 Worker 节点	搭建 K8s 集群，部署 Pod / Service / PVC 等资源
OBS	对象存储服务 OBS (Object Storage Service)	高可用的云端文件存储，类似于网盘/AWS S3，按存储量计费。支持通过 S3 兼容 API 访问	方向 A：存放 Spark 待分析的数据集；也可作为 K8s PVC 的后端存储
ELB	弹性负载均衡 ELB (Elastic Load Balance)	将外部流量分发到多个后端 Pod，提供公网 IP，支持自动健康检查	K8s LoadBalancer 类型 Service 背后的实现，对外暴露应用访问入口

申请步骤：① 注册华为云账号并完成实名认证 → ② "智能基座"入口提交申请（填写课程代码 SCAI004712）→ ③ 代金券到账后按需开通上述服务 → ④ 实验结束后释放 ECS 节点和 ELB，避免超额。

1.4 提交与答辩说明

- 截止时间：期末考试前两周（由教师在课程群通知具体日期）。
- 提交方式：将实验报告（PDF）和代码仓库链接（GitHub/Gitee）发送至助教邮箱，邮件主题格式：【云计算课设】学号_姓名。
- 分组：允许独立或 2 人组队，组队须在报告中注明各人分工比例。

注：未按时提交报告，相关任务分项扣除 50%。

二、第一部分：云计算平台搭建（50 分）

2.1 任务目标

在华为云上部署一套 K8s 云平台，运行"Flask 后端 API + Redis 数据库"两层 Web 应用，实现容器化、对外暴露、配置分离、数据持久化；任务 6（HPA 弹性伸缩）进一步完成弹性伸缩验证。全部 YAML 模板见附录 A。

任务		分值	依赖脚手架
任务 1 应用容器化		10 分	附录 A-1（Dockerfile 模板）
任务 2 CCE 集群搭建		8 分	—
任务 3 应用部署		12 分	附录 A-2（YAML 模板）
任务 4 持久化存储		10 分	附录 A-3（PVC 模板）
任务 5 ConfigMap Volume 挂载		5 分	附录 A-2（ConfigMap 模板）
任务 6 HPA 弹性伸缩		5 分	附录 A-4（HPA 模板）

2.2 任务 1：应用容器化

10 分

基于附录 A-1 的 Dockerfile 模板，完成镜像构建和推送。

1. 修改 Dockerfile.backend：在 requirements.txt 中加入至少 1 个自选 Python 包（如 pandas、requests 等），保留多阶段构建结构。
2. 修改 Dockerfile.frontend：在 Nginx 默认首页 index.html 中加入本人学号和姓名（用于验收识别）。
3. 在本地（或华为云 ECS）运行 docker compose up，截图验证前后端通信正常（后端日志显示收到请求）。
4. 将两个镜像 push 到 SWR，截图保存 SWR 控制台中镜像列表页面（需含镜像名称和 Tag）。

SWR 推送命令格式（具体 Region 和组织名称以控制台显示为准）：

```
docker login -u cn-north-4@<AK> -p <SK> swr.cn-north-4.myhuaweicloud.com
docker tag backend:v1 swr.cn-north-4.myhuaweicloud.com/<ORG>/backend:v1
docker push swr.cn-north-4.myhuaweicloud.com/<ORG>/backend:v1
```

2.3 任务 2：CCE 集群搭建

8 分

通过华为云 CCE 控制台创建 Kubernetes 集群（建议选用 2 vCPU / 4GB ECS 节点以节省代金券额度，后续 CCE 集群不足可新添一个 2 vCPU / 8GB ECS 节点）：

- 集群版本：≥ 1.27，网络插件选 Yangtze CNI（默认即可）。
- 节点数：Master 节点由华为云托管（无需计费），创建 2 个 Worker 节点。
- 配置 kubectl：在 CCE 控制台下载 KubeConfig 文件，配置本地 kubectl 或使用 CloudShell。

验收要求：截图 `kubectl get nodes -o wide`，所有 Worker 节点 Status 为 Ready；截图包含版本信息列（VERSION）。

2.4 任务 3：应用部署

12 分

参考附录 A-2 的 YAML 模板，将任务 1 的镜像部署到 CCE 集群并对外暴露。需完成以下资源（每类各一份 YAML 文件）：

资源	说明	关键要求
Deployment（后端）	运行 Flask API	副本数=2；设置 <code>resources.requests</code> 和 <code>.limits</code> ； <code>configMapRef</code> 注入 Redis 地址； <code>secretKeyRef</code> 注入 Redis 密码
Deployment（Redis）	运行 Redis	副本数=1； <code>limits.memory</code> ≤ 512Mi
Service（后端）	对外暴露	类型 LoadBalancer；添加 <code>kubernetes.io/elb.class: union</code> 注解绑定华为云 ELB
Service（Redis）	集群内访问	类型 ClusterIP；仅内部可达
ConfigMap	非敏感配置	REDIS_HOST = redis-svc, REDIS_PORT = 6379
Secret	敏感配置	Redis 密码用 base64 编码后填入 <code>data</code> 字段（命令： <code>echo -n "pwd" base64</code> ）

验收要求：截图 `kubectl get pods`（所有 Pod Running）；通过 ELB 公网 IP 访问 `/api/ping` 返回 `{"status": "ok"}`（截图浏览器或 curl 结果）。

2.5 任务 4：持久化存储

10 分

为 Redis 配置 PVC，确保 Pod 重建后数据不丢失。参考附录 A-3。

5. 创建 PVC（storageClassName 选华为云 csi-disk）并修改 Redis Deployment，将 /data 挂载到该 PVC。
6. 验证持久化：向 Redis 写入测试数据（redis-cli SET testkey "hello"）并截图；执行 `kubectl delete pod <redis-pod-name>` 触发 Pod 重建；重建完成后查询 GET testkey，截图证明返回 "hello"。

验收要求：提供 `kubectl get pvc`（Status: Bound）截图；写入→删除 Pod→查询的前后对比截图（3 张）。

2.6 任务 5: ConfigMap Volume 挂载

5 分

在任务 3 基础上，将 Nginx 的反向代理配置（upstream 后端地址）改为以 ConfigMap 的 Volume 形式挂载，而非环境变量。

7. 新建一个 ConfigMap，data 中包含 nginx.conf 的完整内容（反向代理到后端 Service）。
8. 修改前端 Deployment，将该 ConfigMap 以 volume 形式挂载到 /etc/nginx/conf.d/default.conf。
9. 修改 ConfigMap 中后端端口值（如 5000→5001），`kubectl apply` 后 `exec` 进前端 Pod，`cat /etc/nginx/conf.d/default.conf` 验证文件已更新，截图。
10. 在报告中用 2–3 句话说明 Volume 挂载与 envFrom 两种方式的适用场景差异。

2.7 任务 6: HPA 弹性伸缩

5 分

为后端 Deployment 配置 HPA，并用压测证明弹性伸缩生效。参考附录 A-4。

11. 确认 metrics-server 可用（`kubectl top nodes` 有数据），CCE 默认已启用，若无数据等待 3 分钟后重试。
12. 创建 HPA：minReplicas=1，maxReplicas=4，targetCPUUtilizationPercentage=60。
13. 用 ab 工具向 ELB 公网 IP 发起压测（持续约 2 分钟）：

```
ab -n 10000 -c 200 http://<ELB_IP>/api/ping
```
14. 开启 `kubectl get pods -w` 监控窗口，截图记录 Pod 数量从 1 增加到 2 或更多的时刻。
15. 停止压测，等待约 5 分钟观察 Pod 数缩回 1，截图记录。
16. 报告中分析：① 扩容延迟的原因（metrics 采集周期 + HPA 评估间隔）；② 冷却时间的意义（防止抖动）；③ HPA 对降本的价值。

提示：若 ab 未安装，可用 Python locust 或 curl 脚本代替；若压测期间 HPA 未触发，在报告中提供排查日志（`kubectl describe hpa`）并分析原因，视分析质量酌情给分。

三、第二部分：并行编程实战（40 分）

3.1 方向选择

在第一部分搭建的 CCE 集群上，选择以下一个方向完成并行编程任务。两个方向满分均为 40 分，YAML 模板见附录 B。

方向	核心技术	适合人群	对应课程
方向 A: Spark 大数据分析	PySpark + Spark Operator	偏数据处理、机器学习	第 9–10 讲
方向 B: MPI 并行科学计算	mpi4py + MPI Operator	偏数值计算、算法	第 9 讲

3.2 方向 A: Spark 大数据分析

A-0. 环境部署（10 分）

10 分

使用附录 B-1 的 SparkApplication 模板，在 CCE 上通过 Spark Operator 提交作业（Helm Chart 离线包由教师课前上传至课程共享目录，或发至课程群）。

17. 安装 Spark Operator: `helm install spark-op ./spark-operator-chart/ -n spark-operator --create-namespace`
18. 修改附录 B-1 的 `sparkapplication.yaml`: 将 `image` 字段替换为教师提供的 SWR PySpark 镜像地址, `executorInstances=2`, `executorMemory="1g"`。
19. 提交 `wordcount.py` 示例作业（附录 B-1），等待 Driver Pod 状态变为 Completed，截图 `kubectl get pods -n default`（含 Driver 和 Executor Pod）。

A-1. 数据清洗（10 分）

从以下数据集中选一个（教师已上传至华为云 OBS，可通过 `s3a://` 接口读取，具体 Bucket 路径在课程群公告中），完成数据清洗：

- 豆瓣电影评分数据集（约 200 MB，含电影 ID、片名、评分、类型、年份等字段）。
- 北京共享单车骑行数据（约 300 MB，含起止站点、时长、用户类型、日期等字段）。

20. 加载数据到 `DataFrame`，打印 `Schema` 和前 5 行；统计各字段缺失值比例。
21. 对至少 2 个有缺失值的字段采用不同处理策略（如 `dropna`、`fillna`），说明选择原因。
22. 输出清洗前后行数对比及各字段基本统计信息（`mean/std/min/max`）。

A-2. Spark SQL 统计分析（15 分）

15 分

使用 Spark SQL 或 DataFrame API 完成至少 4 个统计查询，要求包含：GROUP BY 聚合、ORDER BY Top-N、时间维度趋势分析（按年/月）、JOIN 操作或窗口函数各至少 1 个。每个查询附结果截图及不少于 50 字的分析说明。

A-3. 性能对比与 Amdahl 分析（5 分）

5 分

选取 A-2 中一个查询，分别用 Pandas（单机）和 PySpark（`executorInstances=1` 及 `2`）实现，记录执行时间，绘制对比图；结合 Amdahl 定律分析加速比未达到线性的原因（通信开销、序列化、数据量等）。

3.3 方向 B：MPI 并行科学计算

B-0. 环境部署（10 分）

10 分

使用附录 B-2 的 MPIJob 模板，在 CCE 上部署 MPI Operator 并运行示例作业（Operator YAML 离线包由教师发至课程群）。

23. `kubectl apply -f mpi-operator.yaml` 部署 MPI Operator。
24. 修改附录 B-2 的 `mpijob.yaml`：image 替换为教师提供的 `mpi4py` SWR 镜像，`slotsPerWorker=2`，`workerReplicas=2`。
25. 提交 `pi_mpi.py` 示例作业（附录 B-2），截图展示 Launcher Pod 完成后的日志输出（包含估算的 π 值）。

B-1. 并行算法实现（10 分）

从以下三题选一，实现串行版和 MPI 并行版：

题目	串行算法要点	MPI 通信要点	
题目 1 并行矩阵乘法	三重循环 $O(n^3)$	Scatter 分发行块，Gather 收集结果	
题目 2 数值积分（梯形法）	均分区间累加	Scatter 子区间，Reduce 求和	
题目 3 并行排序（奇偶换序）	冒泡排序	相邻进程比较与交换	

要求：代码中对每个 MPI 通信原语添加注释说明数据流向；并行版结果与串行版一致（截图对比输出）；在报告中附通信模式示意图（手绘拍照可接受）。

B-2. 性能测试与 Amdahl 分析（15 分）

15 分

固定问题规模（如矩阵 $N=800$ 或积分区间 10^7 个点），在 1、2、4 个 MPI 进程下各运行 3 次取平均，填写如下表格并绘制“实测加速比 vs Amdahl 理论加速比”双折线图：

进程数 p	平均运行时间 $T(p)$ / s	实测加速比 S	Amdahl 理论值 (f 待估算)
1		1.00	1.00
2			
4			

根据实测数据估算可并行比例 f ，分析实测与理论的差距原因（通信开销、进程同步等）。

B-3. 非阻塞通信优化（5 分）

5 分

将 B-1 中至少一处关键通信改为非阻塞模式（`comm.Isend / comm.Irecv`），实现计算与通信重叠。对比 4 进程下阻塞版与非阻塞版的执行时间（截图），并在报告中分析：非阻塞通信何时有效、何时改善有限（从网络延迟和计算量的比例角度分析）。

四、附加题（最高 +15 分，自愿完成）

以下三题各 +5 分，独立计分，总计最多加 15 分。未完成不影响正常得分。

附加题 1：监控系统（+5 分）

附加题 +5 分

使用 kube-prometheus-stack Helm Chart（离线包由教师提供）在 CCE 集群部署 Prometheus + Grafana。

- Grafana Dashboard 须展示：节点 CPU 利用率折线图、各 Pod 内存使用柱状图（截图）。
- 报告中说明 Prometheus Pull 采集原理、Grafana 中至少 3 个指标的含义。

评分：+2 分（部署成功截图）+ 2 分（Dashboard 截图 + 指标说明正确）+ 1 分（报告分析深度）

附加题 2: CI/CD 流水线 (+5 分)

附加题 +5 分

为第一部分应用搭建"代码提交 → 自动构建镜像 → 推送 SWR → 更新 K8s Deployment"端到端流水线（GitHub Actions 或 GitLab CI 均可）。

- 截图展示流水线各阶段全部 Passed，以及 K8s Deployment 镜像 Tag 已自动更新。
- 报告中解释 CI/CD 持续集成 vs 持续部署的区别，以及 GitOps 的核心理念。

评分: +2 分（流水线运行截图）+ 2 分（镜像更新验证）+ 1 分（概念说明准确）

附加题 3: 前沿专题 (+5 分, 三选一)

附加题 +5 分

选择以下一个方向，结合课程所学进行深度实践，形成不少于 1500 字的专题内容（含截图或代码）：

C-1 分布式 AI 训练（对应第 16 讲）

用 Horovod 或 PyTorch DDP 在 K8s 上以 2 个 Worker Pod 训练 MNIST CNN，对比单机 vs 分布式训练时间；报告中解释 AllReduce 梯度同步机制，区分数据并行与模型并行。

C-2 边缘计算模拟: K3s + MQTT（对应第 14 讲）

在本地虚拟机安装 K3s 模拟边缘节点，用 paho-mqtt 库将模拟传感器数据通过 MQTT Broker 发布到云端 K8s（Redis 存储）；分析 MQTT 协议在弱网环境下的适用性及云边协同延迟挑战。

评分: +2 分（实验可运行，截图完整）+ 2 分（报告内容深度与准确性）+ 1 分（有批判性思考和自己的见解）

五、评分标准

5.1 第一部分评分细则 (50 分)

任务	评分项	细则	分
任务 1 (10)	Dockerfile 修改	多阶段构建完整；添加了自选 Python 包；前端首页含学号姓名	4
	SWR 推送	SWR 控制台截图含镜像名称+Tag	3
	本地联调	docker compose 运行截图（含后端日志）	3

任务	评分项	细则	分
		志)	
任务 2 (8)	节点 Ready	kubectl get nodes 截图, ≥ 2 个 Worker Ready, 含 VERSION 列	6
	集群版本	VERSION ≥ 1.27	2
任务 3 (12)	Deployment 配置	后端副本=2, 含 resources; Redis 副本=1; 镜像来自 SWR	5
	Service 配置	后端 LoadBalancer 可公网访问 /api/ping; Redis ClusterIP	4
	ConfigMap+Secret	ConfigMap 注入 Redis 地址; Secret 注入密码 (无明文)	3
任务 4 (10)	PVC 绑定	kubectl get pvc 截图 Status=Bound	4
	数据持久化	Pod 删除重建后 GET testkey 仍返回正确值, 3 张对比截图	6
任务 5 (5)	Volume 挂载	ConfigMap 以 volume 挂载, exec 验证文件更新	3
	原理说明	报告中准确说明 Volume 挂载 vs envFrom 的区别	2
任务 6 (5)	扩容截图	Pod 数从 1 增加的 kubectl get pods -w 截图	3
	缩容+分析	停压测后 Pod 数缩回截图; 报告对延迟/冷却/降本的分析	2

5.2 第二部分评分细则 (40 分)

方向	任务	评分要点	分
A	A-0 环境	Driver + Executor Pod Completed 截图; SparkApplication YAML 参数正确	10
A	A-1 清洗	2 种缺失值处理策略, 输出清洗前后行数对比	10
A	A-2 SQL	4 个查询含 GROUP BY、Top-N、时间维度、JOIN/窗口; 各附截图+50 字分析	15
A	A-3 性能	Pandas vs PySpark 对比图; Amdahl 分析有量化讨论	5
B	B-0 环境	Launcher+Worker Pod 截图; π 估算值正确	10
B	B-1 算法	串行/并行结果一致; 通信原语有注释; 附通信图	10
B	B-2 性能	1/2/4 进程时间表; 双折线图; f 估算过程; 差距原因分析	15
B	B-3 优化	改写非阻塞; 对比时间; 分析适用条件	5

5.3 报告质量（10 分）

评分项	细则	分
格式规范	封面含学号/姓名/班级；字体统一；图表有编号和标题	2
内容完整	任务 1-6（或 A-0~A-3 / B-0~B-3）均有步骤说明，无空白截图	4
截图质量	截图清晰；关键信息（Pod 名称/状态/时间/返回值）可辨认	2
分析深度	总结≥200 字，有量化数据，非泛泛描述	2

5.4 总分汇总

模块		满分
第一部分		50 分
第二部分		40 分
报告质量	—	10 分
附加题	3 题各+5 分	最高+15 分
合计		100 分（+15 附加）

六、实验报告结构要求

报告以 PDF 格式提交，建议包含以下章节（无需严格照搬，内容完整即可）：

- 26.封面：课程名、学号、姓名、班级、日期。
- 27.华为云环境信息：账号 Region、CCE 集群版本、节点规格（截图代替描述即可）。
- 28.第一部分实验记录：按任务 1-6 逐项记录，每项含操作步骤摘要 + 关键截图 + 问题与解决方案。
- 29.第二部分实验记录：按所选方向子任务顺序，含代码说明、截图、性能图表。
- 30.总结与收获：对云计算技术的新认识和主要挑战（不少于 200 字）。
- 31.附录：全部修改后的 YAML 文件、Dockerfile 和核心 Python 代码（或 GitHub 仓库链接）。

附录 A：K8s 脚手架模板（第一部分）

以下模板为必填参数已标注 <YOUR_...>，学生需根据自己的华为云账号和应用替换。直接提交未修改的模板视为未完成该任务。

A-1 Dockerfile 模板

Dockerfile.backend (Python Flask, 多阶段构建)

```
# — Stage 1: 安装依赖 —————
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
# 将依赖安装到 /build/packages, 不污染运行时镜像
RUN pip install --no-cache-dir -r requirements.txt --target /build/packages

# — Stage 2: 运行时镜像 (更小) —————
FROM python:3.11-slim
WORKDIR /app
# 只复制依赖包和代码, 不包含 pip/编译工具
COPY --from=builder /build/packages /app/packages
COPY . .
ENV PYTHONPATH=/app/packages
EXPOSE 5000
CMD ["python", "app.py"]

# — requirements.txt 示例 (按需添加其他包) —————
# flask==3.0.0
# redis==5.0.1
# <YOUR_EXTRA_PACKAGE> ← 学生需加入至少 1 个自选包
```

Dockerfile.frontend (Nginx 静态页)

```
FROM nginx:1.25-alpine
# 将自定义 nginx 配置覆盖默认配置
COPY nginx.conf /etc/nginx/conf.d/default.conf
# 将前端静态文件复制到 Nginx 默认目录
COPY static/ /usr/share/nginx/html/
# 注意: static/index.html 中需包含学号和姓名, 用于答辩识别
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

docker-compose.yml (本地联调用)

```
version: "3.9"
services:
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - REDIS_HOST=redis
      - REDIS_PORT=6379
      - REDIS_PASSWORD=${REDIS_PASSWORD:-} # 本地可为空
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

A-2 K8s 核心资源 YAML 模板

deployment.yaml (后端 Flask API)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: default
spec:
  replicas: 2                                # ← 保持 2，不要修改
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: swr.cn-north-4.myhuaweicloud.com/<YOUR_ORG>/backend:v1    # ← 替换
          ports:
            - containerPort: 5000
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          envFrom:
            - configMapRef:
                name: backend-config    # 引用下方 ConfigMap
          env:
            - name: REDIS_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: redis-secret    # 引用下方 Secret
                  key: password
          livenessProbe:
            httpGet:
              path: /api/ping
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 15
```

service.yaml (后端 LoadBalancer + Redis ClusterIP)

```
# — 后端 Service (对外暴露, 绑定华为云 ELB) —————
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: default
  annotations:
    kubernetes.io/elb.class: union    # 华为云必填注解
spec:
  type: LoadBalancer
  selector:
    app: backend
  ports:
    - port: 80
```

```

    targetPort: 5000
---
# — Redis Service (集群内部访问) —————
apiVersion: v1
kind: Service
metadata:
  name: redis-svc
  namespace: default
spec:
  type: ClusterIP
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379

```

configmap.yaml + secret.yaml

```

# — ConfigMap (非敏感配置) —————
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: default
data:
  REDIS_HOST: "redis-svc"
  REDIS_PORT: "6379"
  APP_ENV: "production"
---
# — Secret (敏感配置, 密码需 base64 编码) —————
# 编码命令: echo -n "your_password" | base64
apiVersion: v1
kind: Secret
metadata:
  name: redis-secret
  namespace: default
type: Opaque
data:
  password: <YOUR_BASE64_ENCODED_PASSWORD> # ← 替换

```

A-3 PVC 模板（持久化存储）

```

# — PVC (申请 10Gi 华为云 EVS 云硬盘) —————
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: redis-data-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: csi-disk # 华为云 EVS StorageClass
  resources:
    requests:
      storage: 10Gi

# — 在 Redis Deployment 中挂载 PVC —————
# 在 containers[0] 下加入:
  volumeMounts:
    - name: redis-data

```



```

    mountPath: /data
  volumes:
  - name: redis-data
    persistentVolumeClaim:
      claimName: redis-data-pvc

```

A-4 HPA 模板（弹性伸缩）

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 1
  maxReplicas: 4
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60    # CPU 利用率超过 60% 时触发扩容

# 验证命令（分别在两个终端窗口运行）：
# 窗口 1（监控）：kubectl get pods -w
# 窗口 2（压测）：ab -n 10000 -c 200 http://<ELB_IP>/api/ping

```

附录 B：并行编程脚手架（第二部分）

B-1 方向 A：Spark on K8s 模板

sparkapplication.yaml（SparkApplication CR）

```

apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: pyspark-analysis
  namespace: default
spec:
  type: Python
  pythonVersion: "3"
  mode: cluster
  # ← 替换为教师提供的 SWR PySpark 镜像地址
  image: swr.cn-north-4.myhuaweicloud.com/<YOUR_ORG>/pyspark:3.4
  mainApplicationFile: local:///opt/spark/work/analysis.py
  sparkVersion: "3.4.0"
  restartPolicy:
    type: Never
  driver:
    cores: 1
    memory: "1g"

```

```

    serviceAccount: spark
  executor:
    cores: 1
    instances: 2          # ← 可修改为 1 做性能对比
    memory: "1g"

```

wordcount.py (PySpark 入门示例, 作业提交验证用)

```

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("WordCount").getOrCreate()

# 读取示例文本 (OBS 路径由教师提供)
lines = spark.sparkContext.textFile("s3a://<BUCKET>/sample.txt")

word_counts = (
    lines.flatMap(lambda line: line.split())
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a + b)
    .sortBy(lambda x: x[1], ascending=False)
)

print("Top 10 words:", word_counts.take(10))
spark.stop()

```

B-2 方向 B: MPI on K8s 模板

mpijob.yaml (MPIJob CR)

```

apiVersion: kubeflow.org/v1
kind: MPIJob
metadata:
  name: mpi-pi
  namespace: default
spec:
  slotsPerWorker: 2
  runPolicy:
    cleanPodPolicy: Running
  mpiReplicaSpecs:
    Launcher:
      replicas: 1
      template:
        spec:
          containers:
            - name: launcher
              # ← 替换为教师提供的 SWR mpi4py 镜像地址
              image: swr.cn-north-4.myhuaweicloud.com/<YOUR_ORG>/mpi4py:latest
              command:
                - mpirun
                - -n
                - "4"          # 总进程数 = slotsPerWorker × workerReplicas
                - python
                - /opt/mpi/pi_mpi.py
    Worker:
      replicas: 2          # ← 调整此值做多进程性能测试
      template:
        spec:
          containers:
            - name: worker

```

```
image: swr.cn-north-4.myhuaweicloud.com/<YOUR_ORG>/mpi4py:latest
resources:
  requests:
    cpu: "1"
    memory: "1Gi"
```

pi_mpi.py (蒙特卡洛估算 π , 入门示例, 作业提交验证用)

```
from mpi4py import MPI
import random

comm = MPI.COMM_WORLD
rank = comm.Get_rank()      # 当前进程编号
size = comm.Get_size()      # 总进程数

N = 10_000_000               # 每个进程各采样 1000 万次
local_count = 0
for _ in range(N):
    x, y = random.random(), random.random()
    if x*x + y*y <= 1.0:
        local_count += 1

# Reduce: 将所有进程的 local_count 汇总到 rank=0
total = comm.reduce(local_count, op=MPI.SUM, root=0)

if rank == 0:
    pi = 4.0 * total / (N * size)
    print(f"[{size} processes]  $\pi \approx {pi:.6f}$ ")
```